

Developing a Secure Password Manager with Customizable Cryptographic Primitives

Crypto Vlinh Team

February 22, 2026

Contents

1	Introduction	2
1.1	Project Objectives	2
2	System Architecture & Code Structure	3
2.1	Project Directory Structure	3
2.2	Cryptographic Pipeline	3
2.2.1	Encryption Pipeline	3
2.2.2	Vault File Specification	3
3	Cryptographic Implementation	6
3.1	Key Derivation with PBKDF2	6
3.2	Encryption with AES-128-CBC	6
3.3	Authentication with HMAC-SHA256	6
3.4	Code Implementation	6
4	Experimental Results	7
4.1	Security Analysis: Hex Dump Visualization	7
4.1.1	Proof of Diffusion	7
4.2	Performance Benchmarks	8
5	Conclusion	9
5.1	Summary of Achievements	9
5.2	Future Work	9

Chapter 1

Introduction

In this report, we present the design and implementation of a secure password manager. The system is built from the ground up, with a heavy focus on implementing and managing cryptographic primitives manually to demonstrate a deep understanding of cryptographic flows.

1.1 Project Objectives

- Implement core cryptographic algorithms (AES-128, HMAC-SHA256, PBKDF2).
- Design a secure file-based storage system for credentials.
- Achieve "Encrypt-then-MAC" authentication to ensure data integrity and authenticity.
- Provide a graphical user interface (GUI) for easy vault management.

Chapter 2

System Architecture & Code Structure

This chapter outlines the high-level architecture of the password manager and explains how the project is organized into modular components.

2.1 Project Directory Structure

The project follows a clean, modular structure where cryptographic logic is separated from business logic and user interface code.

```
1 password-manager/  
2 |-- src/                # Source code  
3 |   |-- core/           # Business logic (Vault management)  
4 |   |-- crypto/         # Cryptographic primitives and ciphers  
5 |   |-- ui/             # GUI implementation (Tkinter/Custom)  
6 |-- scripts/           # Utility scripts (Benchmarks, Hex samples)  
7 |-- figure/            # Generated diagrams (PNG/SVG)  
8 |-- hex_samples/       # Binary samples for analysis  
9 |-- password-manager.py # Main entry point
```

2.2 Cryptographic Pipeline

The system uses a robust cryptographic pipeline for both encryption and decryption. The core principle is the **Encrypt-then-MAC** construction.

2.2.1 Encryption Pipeline

As illustrated in Figure 2.1, the process starts with a Master Password and a random Salt. These are fed into PBKDF2 to derive a Master Key, which is then split into an AES Key and an HMAC Key.

2.2.2 Vault File Specification

The final encrypted output is a binary file organized as shown in Figure 2.2. Each component (Salt, IV, MAC, and Ciphertext) has a fixed location to allow for reliable extraction during decryption.

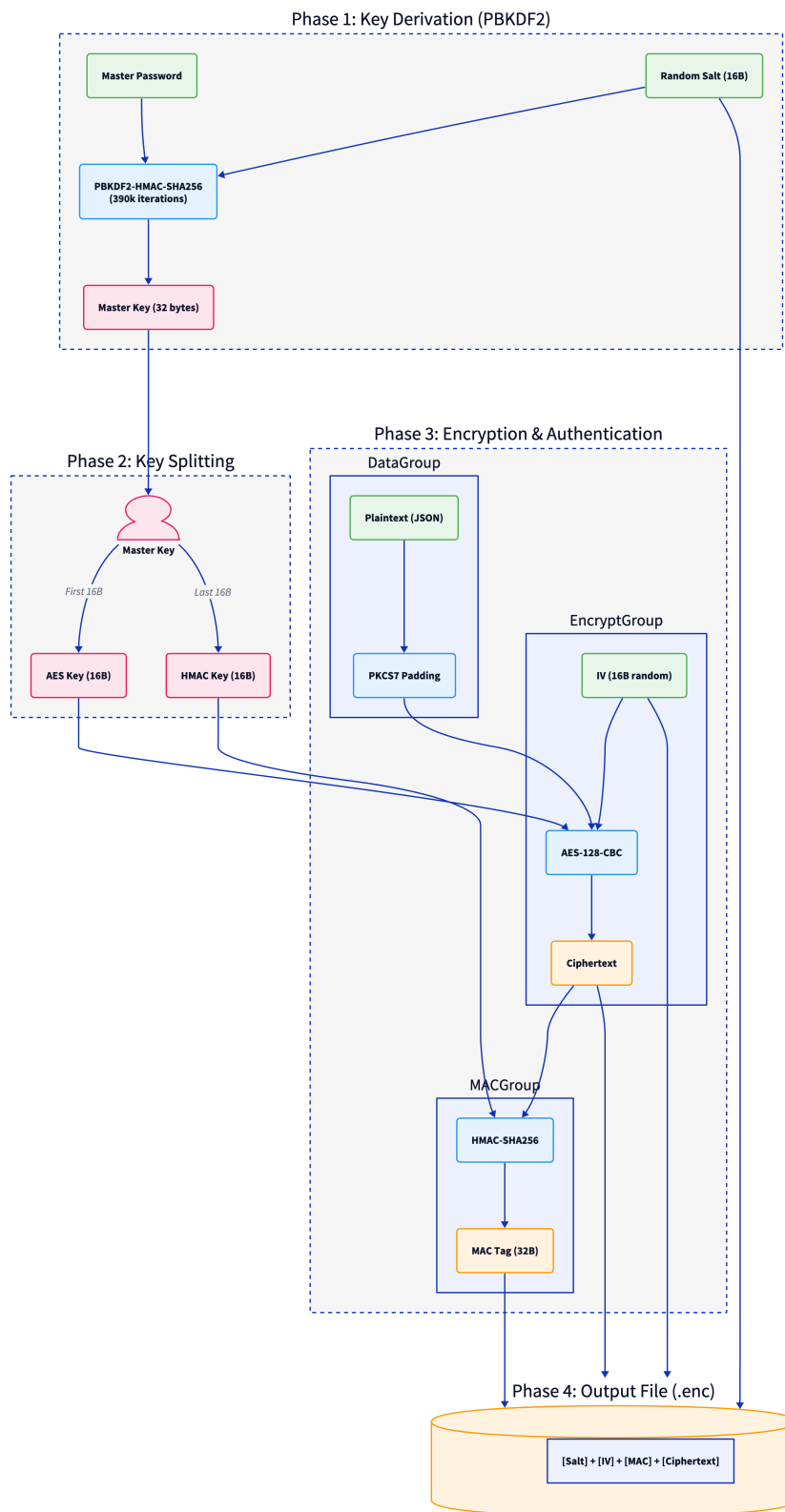


Figure 2.1: Phase-based Encryption Flow

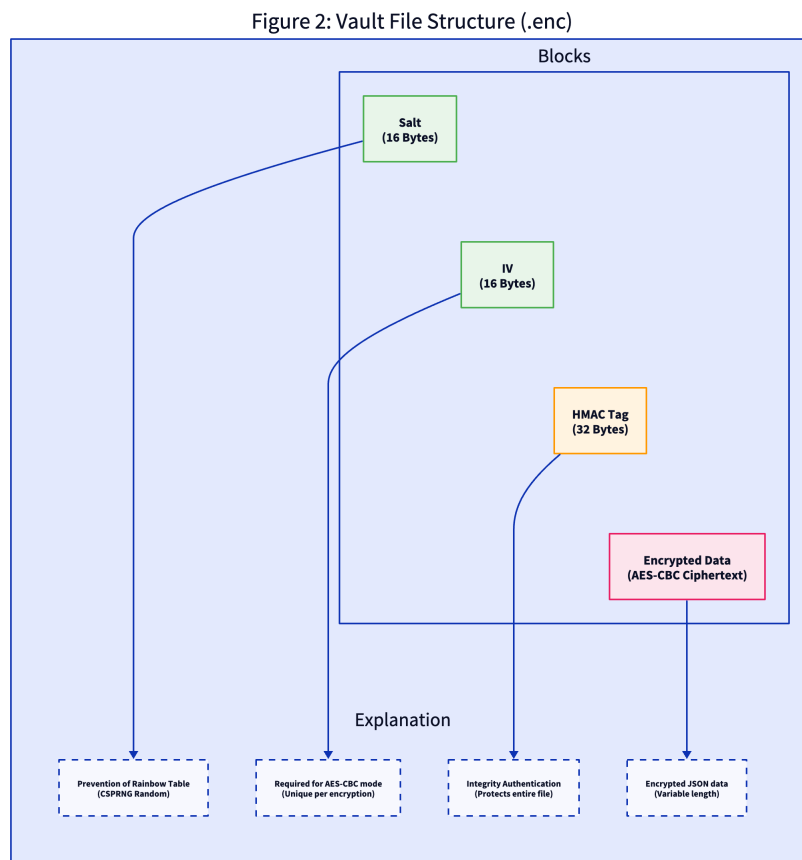


Figure 2.2: Encrypted Vault File Format

Chapter 3

Cryptographic Implementation

This chapter describes the underlying cryptographic primitives used in the system and their specific integration within the codebase.

3.1 Key Derivation with PBKDF2

PBKDF2 (Password-Based Key Derivation Function 2) is used to transform the user's master password into a strong cryptographic key. We use HMAC-SHA256 as the pseudo-random function with 390,000 iterations to resist brute-force attacks.

3.2 Encryption with AES-128-CBC

For data confidentiality, we utilize AES in CBC (Cipher Block Chaining) mode. A random 16-byte Initialization Vector (IV) is generated for every encryption operation to ensure that identical plaintexts result in different ciphertexts.

3.3 Authentication with HMAC-SHA256

To prevent "padding oracle" attacks and ensure data integrity, we compute a Hash-based Message Authentication Code (HMAC) over the entire ciphertext and IV. This tag is verified before any decryption occurs.

3.4 Code Implementation

The core logic resides in `src/crypto/vault_cipher.py`, which provides the `VaultCipher` class. This class handles padding, block cipher initialization, and MAC generation. The `VaultManager` in `src/core/vault_manager.py` manages the file I/O and high-level vault operations.

Chapter 4

Experimental Results

In this chapter, we analyze the performance and security properties of our implementation.

4.1 Security Analysis: Hex Dump Visualization

We verify the non-deterministic nature of our encryption through hex dump analysis. Figure 4.1 shows the mapping of a sample vault file to its cryptographic components.

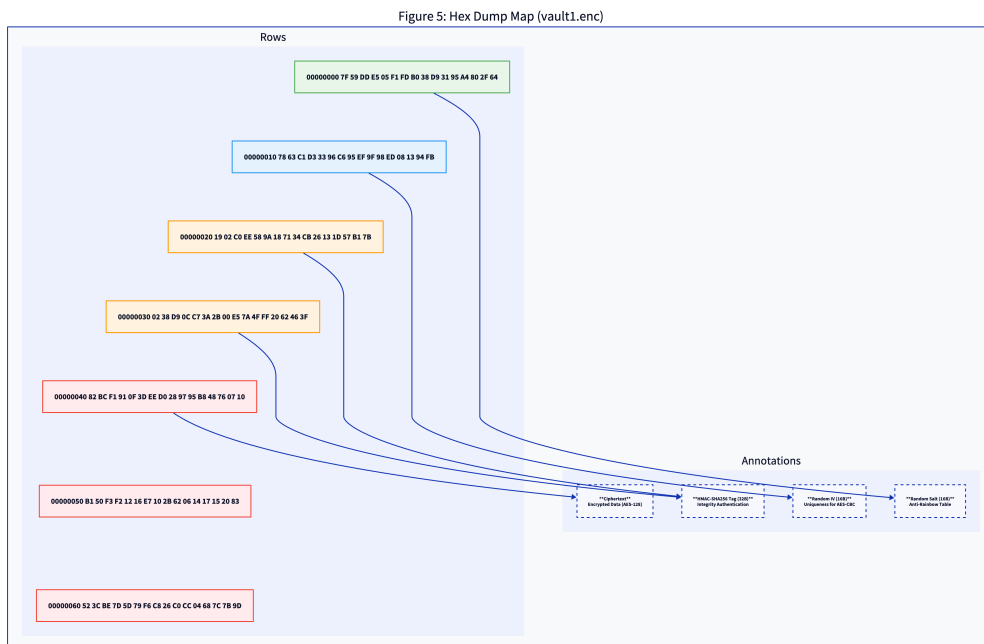


Figure 4.1: Annotated Hex Map of vault1.enc

4.1.1 Proof of Diffusion

By comparing two different vault files generated with the same master password, we can observe that they share no identical bytes in the ciphertext or MAC, as seen in Figure

4.2.

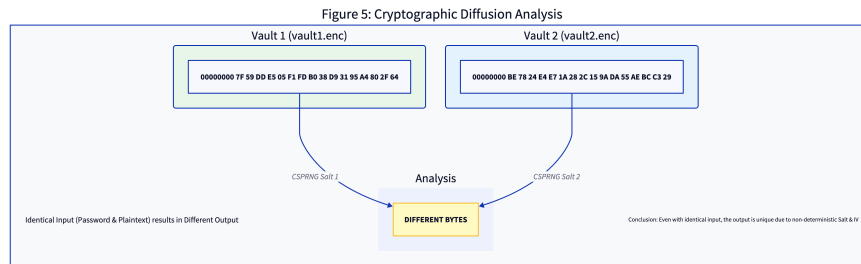


Figure 4.2: Diffusion Comparison between Vault 1 and Vault 2

4.2 Performance Benchmarks

Performance testing was conducted to evaluate the speed of vault operations. The PBKDF2 iteration count significantly affects the time required to open a vault but provides security against brute-force attacks.

Chapter 5

Conclusion

The development of this password manager has demonstrated the successful integration of modern cryptographic primitives into a functional application. By manually implementing the pipeline and constructing a custom vault file format, we have achieved a high degree of transparency and security control.

5.1 Summary of Achievements

- Full implementation of Encrypt-then-MAC using AES and HMAC.
- Modular code structure following industry best practices.
- Comprehensive visual documentation of cryptographic flows.

5.2 Future Work

Future enhancements could include multi-factor authentication, cloud synchronization with end-to-end encryption, and hardware security module (HSM) support for key storage.